

Part B [Team work]

Task 3

3.1 Requirements:

a. Functional Requirements:

1. The Application should be able to accept and process stateless Data stream from sets of registered IOT devices.
2. In real time, the Application would format the data into a stunning visual dashboard.
3. The Application should execute Predefined Queries
4. IS3 would be able to Authorize and Authenticate Users access.

b. Non-Functional Requirements

1. Usability for User Experience. The front-end must be user-friendly, with minimal learning curve, and optimized for both web and mobile (iOS and Android).
2. Extensibility/Flexibility. Allow easy integration of new IoT devices, additional features, or external services without significant changes to the core system.
3. Interoperability. Support a variety of IoT devices and standards (e.g., MQTT protocol for device communication).
4. Scalability. Capability of handling an increasing number of devices and users without performance degradation.
5. Availability. The system should ensure 99.99% uptime through high availability architectures (e.g., multi-AZ deployments, failover mechanisms).
6. Reliability & Disaster Recovery. Ability to handle device communication failures gracefully and provide fallback mechanisms (e.g., retries, logging of errors). Also for the architecture it is obligatory to have a robust disaster recovery plan, including automated backups, replication, and quick failover.
7. Data Consistency & Integrity. The system should maintain consistency between the relational metadata and timeseries data, particularly when scaling or in case of a service failure. Also It ensures that the data received from IoT devices is stored correctly in the database without loss or corruption (good serialisation and marshallng).
8. Maintainability of documentation. Well-documented APIs, clear code standards, and centralized logging (ELK stack) for easy debugging and maintenance.

3.2 Components mapping and outline

To ensure our application's scalability, resource and time optimization we decided to design it as a set of microservices, each responsible for a specific portion of the overall task. With this design each service will be able to scale and be developed separately, ensuring high availability, constant non-conflicting updates and high resilience.

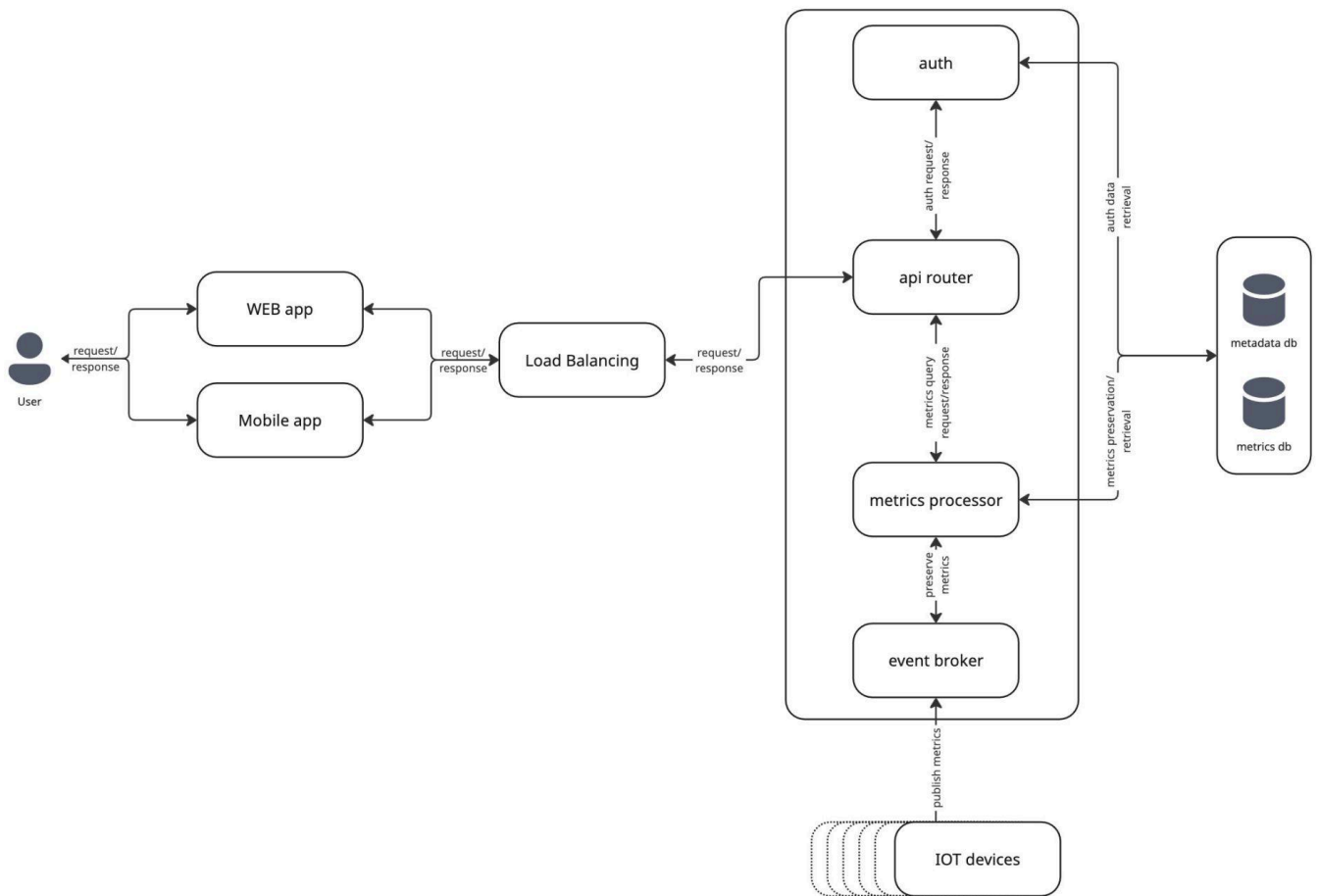
For microservices management an orchestrator will be utilized (i.e. k8s) to automate resource management, scaling and cross-service communication. Most of the services will be hosted in a private subnet, user access will be only allowed through API calls via a gateway, enhancing security and availability in case of DoS attacks. Additionally, to ensure constant service monitoring and alerting a set of internal metric collection and aggregation tools will be used.

Also 2 different types of databases were chosen to handle different aspects of working with application data - regular relational db for auth/meta data and timeseries db for storing and querying iot metrics, which will help to separate querying and dashboard building into a separate logic and enhance performance.

Detailed component description is as follows:

1. Databases
 - a. metadata db (relational db) - stores user info, related device metadata, etc.
 - b. metrics db (timeseries db) - for storing collected metrics
2. User App - a wrapper for calling API endpoints, rendering visuals for returned metrics
 - a. Front-end for web-version
 - b. Mobile App on Android and IOS
3. IOT devices - metrics-providing devices queried by the customer, main data source for dashboards
4. Load balancing - direct incoming requests to appropriate microservices, handle proxying
5. Services Orchestration (k8s) - middleware for managing microservice communication, scaling, etc.:
 - a. M1 (auth) - user authentication and authorization (i.e for scoping - users must only query their devices)
 - b. M2 (api) - endpoints collection for auth, querying devices and dashboard data retrieval
 - c. M3 (broker, MQTT) - iot healthcheck, metrics collection, background topic subscription to known devices for timeseries data collection
 - d. M4 (metrics and dashboards processor) - active/passive (some metrics are calculated before queries, some only on-demand) data storing/processing for visualization
 - e. M5 (monitoring stack) - metrics collection from internal services for alerting and observability.
6. User - end consumer for visual data and device management endpoints

The schematic architecture with all components:



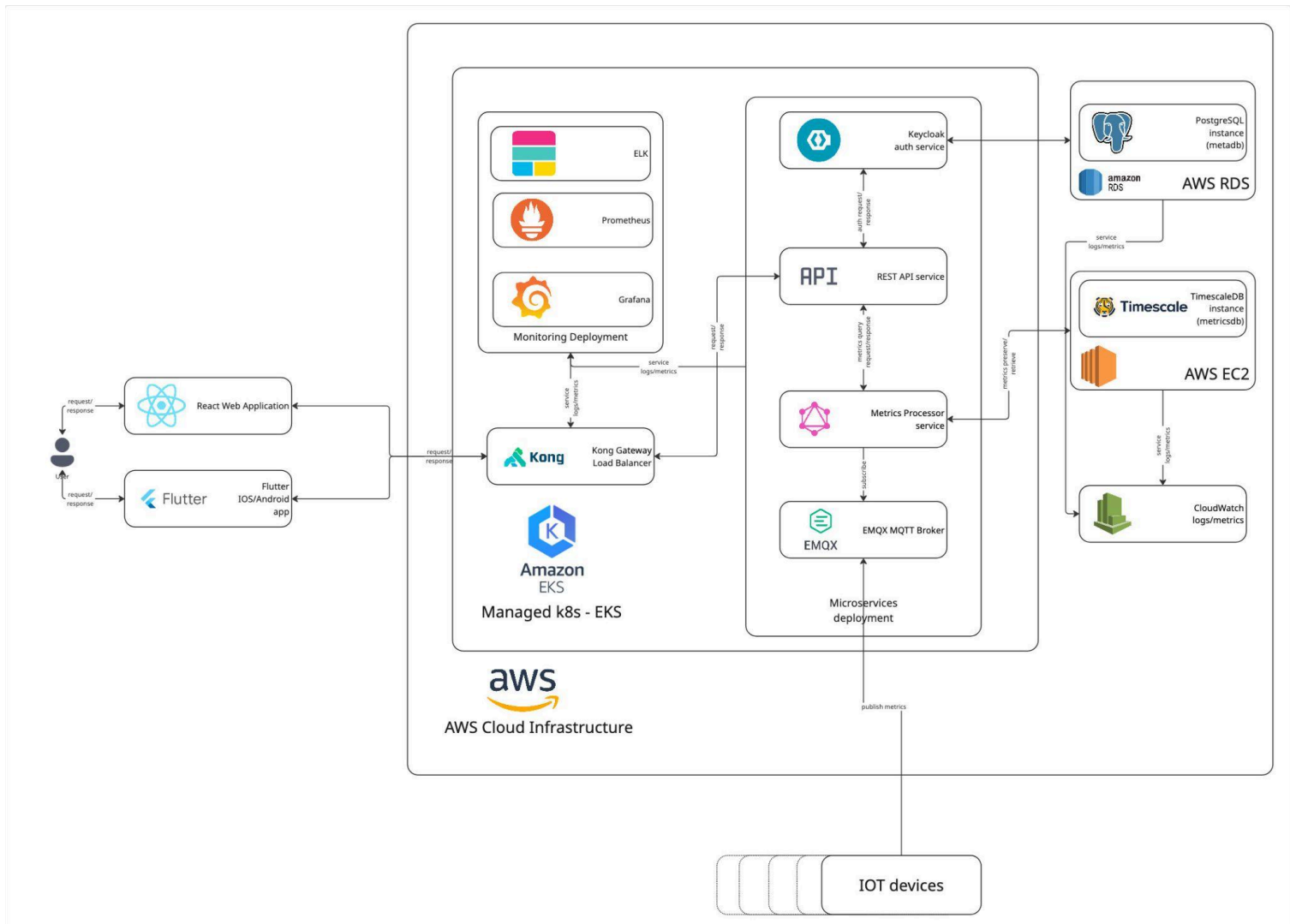
3.3 Technology stack

- Databases:
 - a. metadata db - PostgreSQL for customers, devices and role-based access control data (deployed/configured with AWS RDS).
 - b. metrics db - TimescaleDB for joining metrics with relational metadata (deployed on AWS EC2).
- UserApps:
 - a. Front-end: React - providing high performance and stability, also supports backward compatibility assuring long-time support.
 - b. Mobile app: Flutter is good for compiling high-performance applications, providing high development speed with hot reloads and comprehensive SDK.
- IOT devices communicate with system using MQTT protocol publishing events to broker using mTLS.
- Load balancing: Kong Gateway - is a cloud-native gateway that runs natively on Kubernetes, providing routing, balancing, rate limiting and supports OIDC and mTLS.
- Orchestration: Kubernetes for deployment, scaling, resiliency:
 - a. Auth - Keycloak open-source access management solution that supports Kong OIDC.

- b. API - REST for centralized queries.
- c. Event broker - EMQX MQTT Broker
- d. Metrics processor - service processing events from broker, normalizing and enriching them and writing them into DB. Dashboard - GraphQL is great for data-rich UI and cross-platform apps, reducing complexity.
- e. Monitoring - Prometheus and Grafana for kubernetes metrics (healthchecks) and ELK for logs from services. For DBs deployed outside of k8s, CloudWatch can be used for this purpose.
- The system will be cloud based using Amazon Web Services(AWS) because it has Elastic Kubernetes Service(EKS) that makes it easier to deploy, manage and scale containerized applications, also it has the widest managed service catalog for IoT compatible systems.

3.4 Architecture design

The architecture design with mapped components and chosen software:



3.5 Short ADRs (Architectural Decision Record)

ADR for Databases

We chose PostgreSQL for storing metadata (user info, device data, etc.) due to its reliability, relational nature, and ease of use with AWS RDS. For time-series data, we opted for TimescaleDB to efficiently handle large volumes of IoT metrics, providing better performance for time-dependent queries.

ADR for UserApps

React is one of the most commonly used frameworks for designing modern web-apps. It provides a variety of libs that are essential for Data consistency & Integrity with different services, Scalability and Flexibility of User applications. And also it provides great tools to create really User friendly application. React works very well with other architecture choices, like Rest API, Databases, k8s.

Flutter is used for the mobile app, ensuring high development speed and native performance across both iOS and Android platforms.

ADR for Load Balancing

Kong Gateway is used for load balancing, routing, rate-limiting, and mTLS support, running natively on Kubernetes. It ensures scalability, secure API management, and smooth traffic distribution across microservices.

ADR for Orchestration and AWS

Kubernetes (EKS on AWS) is chosen for orchestrating microservices to automate scaling, resource management, and cross-service communication. It enhances system reliability and allows easy handling of failures and auto-scaling.

By deploying applications across multiple AZs (availability zones), while using AWS, we enhance availability of the service.

ADR for main components

Keycloak provides centralized authentication and authorization through role-based access control. This means each user can only view and manage their own devices. It also integrates seamlessly with the Kong Gateway fitting perfectly into the system.

REST is a strong fit for this IoT system because it provides simple, predictable, and widely adopted HTTP interfaces that work seamlessly with web and mobile clients, and API gateways.

EMQX MQTT broker supports IoT standards and integrates smoothly to Kubernetes. It ensures normalization and enrichment of data optimizing query time for GraphQL dashboards.

Prometheus, Grafana and ELK enabling comprehensive observability, monitoring, logging and alerting.

The result system is a resilient, extensible platform that takes real-time data from multiple IoT devices, keeps it safe and organized, and turns it into fast comprehensive dashboards for people to use. System built in small independent parts, providing convenient scalability. Security and reliability provided via strong access controls, high availability and backups.